

The CdM-16 Extended Instruction Set

Gleb Bagrov

Artem Pisarev

February 01, 2026

Contents

1. Overview	3
1.1. Introduction	3
1.2. Overview	3
2. Implementation	4
2.1. Instruction encoding	4
2.2. Multiplication instructions	5
2.2.1. umul	5
2.2.2. smul	6
2.3. Division instructions	7
2.3.1. udiv	7
2.3.2. sdiv	8
3. Elaboration	9
3.1. Research	9
3.2. Uses	9
3.3. Requirements	9
3.4. Benchmarks	9
3.5. Risks	10
Index of Figures	11

1. Overview

1.1. Introduction

This document presents the current draft of the CdM-16 Modified Instruction Set Manual, Version 1.0, which extends the original CdM-16 architecture with hardware-supported multiplication and division instructions.

The goal of these additions is to enhance computational throughput while keeping the simplicity and compactness of the 16-bit instruction format. The modified instruction set preserves full compatibility with the existing CdM-16 core. The new operations are encoded within the same 16-bit framework, using reserved opcode space. This approach maintains the original design philosophy: a minimal, predictable ISA suitable for educational processors.

These extensions provide more expressive arithmetic capabilities with minimal overhead. The intent is to deliver a stable specification for implementers.

1.2. Overview

The CdM-16 Modified Instruction Set extends the original 16-bit the CdM-16 architecture with compact arithmetic operations, adding hardware-level multiplication and division while preserving compatibility with all existing instructions. The new operations use reserved opcode space and follow the same structural rules as the base ISA.

The CdM-16 instruction format remains fixed at 16 bits, so the modified ISA focuses on efficient encoding strategies. Because the new arithmetic operations follow the same structural conventions, implementers can incorporate them without significant changes to the decode or execute. The added operations allow more computation to occur directly in hardware, reducing the need for multi-instruction sequences in code paths. This improves performance.

The CdM-16 extension ‘M’ keeps all instructions 16-bit. This reduces implementation complexity, ensures deterministic instruction fetch behavior, and preserves the educational clarity of the original design. The modified ISA aims to deliver stronger arithmetic capabilities with minimal architectural overhead, supporting both compact binaries and predictable execution. The goal of these extensions is to provide a stable, minimal, and practical enhancement to the CdM-16 architecture.

2. Implementation

This extension introduces hardware-supported integer multiplication and division into the CdM-16 architecture. The result is an implementation model that is simple to reason about and easy to decode.

2.1. Instruction encoding

All instructions in CdM-16 are encoded in a fixed 16-bit format. The multiplication and division instructions use the standard 2-operand encoding form already present in the base architecture.

The general 2-operand encoding is shown below.

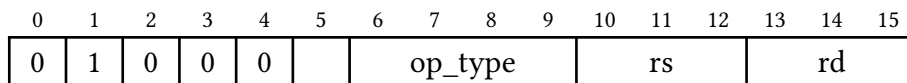


Figure 1: Encoding 2op

- **op_type** A 4-bit opcode uniquely identifying the instruction.
- **rs** Source register.
- **rd** Destination register.

All arithmetic instructions operate purely on registers. Immediate operands are not supported in this encoding. The architecture defines eight 16-bit general-purpose registers, one of which is reserved as the frame pointer.

2.2. Multiplication instructions

Multiplication is implemented in hardware as a multi-cycle, fully deterministic operation. Both signed and unsigned variants are provided. The result always preserves the full 32-bit product of two 16-bit operands, avoiding overflow by construction.

In multiplication instructions, the result is typically split across two registers:

rl (low register): Holds the lower-order bits of the product.

rh (high register): Holds the higher-order bits of the product.

Unlike standard 2-operand instructions that use **rs** (source) and **rd** (destination) terminology, multiplication operations employ **rl** and **rh** as dual-purpose registers—each functioning as both source and destination depending on the architecture and instruction variant.

2.2.1. umul

Unsigned multiplication. Multiplies the contents of registers *rl* and *rh*, storing the lower-order bits in *rl* and the higher-order bits in *rh*.

umul *rl, rh*

Encoding

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	0	0	0	0	1	1	0	0	rl			rh		

Figure 2: Encoding umul

Operation semantics

```
tmp = zero_extend(rl) * zero_extend(rh)

if (num(rl) == num(rh)) { # num(reg) - return ordinal
    rl = tmp[15:0]        #          number of register
} else {
    rl = tmp[15:0]
    rh = tmp[31:16]
}
```

Flags

The instruction updates the condition flags:

- C (carry): Set if **rh** contains non-null value.
- V(overflow): Always cleared (overflow cannot occur in unsigned multiplication).
- Z (zero): Set if the resulting value in the **rl** register is zero.
- N(negative): Always cleared.

Overflow is architecturally impossible due to full-width result storage.

If **rh** and **rl** contain the same register, the result will be low-order bits of multiplication.

2.2.2. smul

Signed Multiplication. Multiplies the contents of registers *rl* and *rh* as signed integers, storing the lower-order bits in *rl* and the higher-order bits in *rh*. Signed operands are interpreted using two's complement representation. The result preserves the full signed 32-bit product.

smul *rl, rh*

Encoding

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	0	0	0	0	1	1	1	0	rl			rh		

Figure 3: Encoding smul

Operation semantics

```
tmp = sign(rl) * sign(rh)
if (num(rl) == num(rh)) { # num(reg) - return ordinal
    rl = tmp[15:0]        #          number of register
} else {
    rl = tmp[15:0]
    rh = tmp[31:16]
}
```

The behavior is identical to `umul` except for operand interpretation and these flags:

- V (overflow): Set if a signed operation produces a result that exceeds the representable range.
- N (negative): Set if the most significant bit of the *rh* is 1, indicating a negative value in two's complement representation.

If `rh` and `rl` contain the same register, the result will be low-order bits of multiplication.

2.3. Division instructions

Division is implemented as a multi-cycle hardware operation. Only integer division is currently defined. The quotient and remainder are both architecturally visible.

The quotient is written to `rq`.

The remainder is written to `remr`.

The rounding mode is intentionally left undefined, emphasizing that software must not rely on a specific rounding direction.

2.3.1. udiv

Unsigned Division. Divides the contents of register *rs* by *remr*, storing the quotient in *rq* and the remainder in *remr*. Division by zero raises an exception.

udiv *rq, remr*

Encoding

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	0	0	0	0	1	1	0	1	rq			remr		

Figure 4: Encoding udiv

Operation semantics

```
if (rq == 0) {
    rq = 0
    remr = 0
} else {
    if (num(rq) == num(remr)) { # num(reg) - return ordinal
        rq = 1                #          number of register
    } else {
        quotient = zero_extend(rq) / zero_extend(remr)
        remainder = zero_extend(rq) % zero_extend(remr)
        rq = quotient
        remr = remainder
    }
}
```

Flags

- C: Always cleared.
- V: Always cleared (overflow cannot occur in unsigned division).
- Z: Set if the quotient is zero.
- N: Always cleared.

If `rq` and `remr` contain the same register, the result will be low-order bits of division (always 1).

2.3.2. sdiv

Signed Division. Divides the contents of register *rq* by *remr* as signed integers, storing the quotient in *rq* and the remainder in *remr*. Division by zero or overflow sets *rq* and *remr* to zero and the carry flag to 1. Operands are interpreted as two's complement signed integers.

sdiv *rq, remr*

Encoding

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	0	0	0	0	1	1	1	1	rq			remr		

Figure 5: Encoding sdiv

Operation semantics

```
if (remr == 0) {
    rq = 0
    remr = 0
    C = 1
} else {
    if (num(rq) == num(remr)){ # num(reg) - return ordinal
        rq = 1                # number of register
    } else {
        quotient = sign(rq) / sign(remr)
        remainder = sign(rq) % sign(remr)

        rq = quotient
        remr = remainder
    }
}
```

A special case exists when dividing the minimum negative value by -1 . In this case, the carry flag is set and the result is forced to zero, preventing an unrepresentable signed result.

`sdiv` behaves identically to `udiv`, except for signed interpretation and flags:

Remainder

`sdiv` always returns positive remainder

Flags

- C: Always cleared.
- V: Set if the signed result exceeds the representable range (minimum negative / -1 case).
- Z: Set if the quotient is zero.
- N: Set if the most significant bit of the quotient is 1, indicating a negative result.

3. Elaboration

3.1. Research

Several historically architectures were studied, including the Motorola 68000, PDP-11, AVR, and the MIL-STD-1750A standard. These architectures demonstrate a wide range of approaches to instruction encoding, arithmetic unit design, and operand handling, particularly with respect to multiplication and division.

The analysis focused on how these systems balance simplicity, explicitness, and performance—especially how they expose multi-word arithmetic results and handle exceptional cases such as division by zero. The resulting CdM-16 design intentionally adopts the most transparent and instructive solutions observed across these architectures, while avoiding unnecessary complexity.

3.2. Uses

The primary purpose of this project is educational. The CdM-16 architecture is intended to serve as a platform for understanding computer architecture, instruction set design, and low-level arithmetic operations.

In its base form, CdM-16 supports fundamental operations such as data movement, addition, and subtraction. By introducing hardware-supported multiplication and division, the architecture becomes capable of expressing a much broader class of programs while remaining conceptually accessible to students.

3.3. Requirements

The newly introduced instructions must integrate seamlessly into the existing CdM-16 instruction set. In particular:

- They must not interfere with or alter the semantics of any existing instructions.
- Their encoding must follow the same design principles and optimization level as the current instruction set.
- All arithmetic instructions must use the standard double-operand format.
- The opcode space used for the extension is limited to the predefined 4-bit opcode field, with values reserved specifically for arithmetic expansion.

Special attention is paid to error conditions. The architecture must define deterministic behavior for exceptional cases such as division by zero and invalid or empty operands, avoiding undefined states or implicit side effects.

For multi-word arithmetic results, the lower-order bytes are stored in `rq`, while the higher-order bytes are stored in `remr`. This convention is consistent across all relevant instructions and is architecturally guaranteed.

3.4. Benchmarks

The performance and correctness of the extended instruction set are evaluated using a focused set of benchmarks:

- Multiplication and division of relatively small integer values, validating basic correctness and flag behavior.
- Multiplication and division of relatively large integer values, ensuring correct handling of full-width results and edge cases.

These benchmarks are designed not only to measure execution characteristics but also to validate architectural guarantees such as determinism and result placement.

3.5. Risks

Several risks are associated with extending the CdM-16 instruction set:

- Compatibility risks with existing instructions, particularly regarding opcode allocation and decoding. These risks are mitigated by incremental integration and continuous testing.
- Increased architectural complexity, which may reduce clarity if not carefully constrained.
- Performance implications arising from multi-cycle multiplication and division instructions, potentially introducing pipeline stalls or hazards.
- Pipeline hazards specific to long-latency arithmetic operations, requiring careful consideration during implementation.
- The possibility of bugs or malfunctions introduced by the new instructions, emphasizing the need for thorough testing and verification.
- Additional challenges in validation and verification as the instruction set grows.

Despite these risks, the controlled and minimal nature of the extension ensures that CdM-16 remains a coherent and approachable architecture, even as its expressive power increases.

Index of Figures

Figure 1	Encoding 2op	4
Figure 2	Encoding umul	5
Figure 3	Encoding smul	6
Figure 4	Encoding udiv	7
Figure 5	Encoding sdiv	8